

Improving Micro-Expression Recognition using Temporal and Motion Modelling

A modular deep-learning pipeline on CASME-II combining motion magnification, optical-flow/strain motion representation, 3D-CNN spatio-temporal features, parameter-free attention and a sequence transformer.

Author: Addhyan

► SPEAKER NOTES — what to say

Good morning/afternoon. My thesis is titled *Improving Micro-Expression Recognition using Temporal and Motion Modelling*. The core idea is simple to state: micro-expressions are defined by tiny, fast facial *motion*, so if I want a machine to recognise them I have to (a) model that **motion** explicitly, and (b) model how it evolves over **time**. Everything in this presentation is organised around those two words — *motion modelling* and *temporal modelling*.

Over the next slides I will explain the problem, then walk through every single technology I used — Eulerian Video Magnification, optical flow and optical strain, a 3D-CNN, the SimAM attention module, and a sequence transformer — one at a time. I will then show how I combined all of them into my final 'proposed unified' model, describe each experiment step by step, present the results with the actual figures, and finish with a comparison table and conclusions.

The headline finding, which I'll build up to honestly, is that motion modelling and spatial attention carried the performance, while the temporal transformer — although theoretically attractive — actually hurt on a dataset this small. That nuance is the scientific contribution.

What is a micro-expression — and why is it hard?

- **Micro-expressions (MEs)** are brief ($\approx 1/25$ – $1/2$ s), involuntary, low-intensity facial movements that leak a person's *genuine* emotion even when they try to hide it.
- Applications: clinical psychology, deception/▶security screening, affective computing, human-computer interaction.
- Why they are hard to recognise:
 - **Short** — only a handful of frames carry the signal.
 - **Subtle** — motion is a few pixels of skin movement.
 - **Buried** — under identity, lighting and head-pose variation.
- Key insight driving this thesis: the discriminative information is **motion over time**, not static appearance.

▶ SPEAKER NOTES — what to say

Let me set up the problem. A micro-expression is a very brief, involuntary facial movement — typically lasting between a twenty-fifth and a half of a second — that reveals an emotion a person is consciously trying to suppress. Because they are genuine and hard to fake, they matter in psychology, security screening, lie detection research, and affective computing.

But they are extremely hard for a computer to detect, for three reasons. They are *short* — the signal lives in only a few video frames. They are *subtle* — we are talking about a few pixels of skin movement, not a big grin. And they are *buried* — the same video also contains the person's identity, the room lighting, and head motion, all of which are far stronger signals than the micro-expression itself.

The single most important observation — and the reason for my thesis title — is that the useful information is *motion that unfolds over time*. A still image is almost useless. So the entire system is designed to first make that motion visible and explicit (motion modelling), and then to model how it evolves frame to frame (temporal modelling).

Thesis goal: model motion AND time, then prove which parts help

- **Thesis statement:** combine *motion modelling* and *temporal modelling* into one pipeline and measure the contribution of each component.
- **Motion modelling** = make the tiny movement visible and represent it: Eulerian magnification → optical flow + optical strain → 3D-CNN.
- **Temporal modelling** = capture how the movement evolves: 3D-CNN temporal convolutions (short-range) + a sequence transformer (long-range onset→apex→offset).
- **Success criteria (3-class grouped, CASME-II):** Accuracy ≥ 0.70 and Macro-F1 ≥ 0.68 , under subject-disjoint Holdout and Leave-One-Subject-Out (LOSO).
- **Method:** a full ablation — switch each technology on/off — so every claim is evidence-based, not assumed.

► SPEAKER NOTES — what to say

Here is the thesis statement in one line: I build a single pipeline that does both motion modelling and temporal modelling, and then I rigorously measure how much each individual component actually contributes — rather than just assuming a bigger model is better.

When I say *motion modelling* I mean three things working together: Eulerian Video Magnification to amplify the tiny movement, optical flow and optical strain to turn that movement into a numerical representation, and a 3D-CNN to extract local motion features. When I say *temporal modelling* I mean capturing how that motion changes across the 32 frames — the 3D-CNN does this at short range, and a transformer does it at long range, capturing the full onset-to-apex-to-offset trajectory.

My success criteria were fixed in advance from the literature: on the 3-class grouped problem I wanted at least 70% accuracy and a macro-F1 of at least 0.68, validated both with a fast Holdout split and with the stricter leave-one-subject-out protocol. And critically, my method is an *ablation study*: I turn each technology on and off in a controlled matrix, so by the end I can say exactly which pieces earned their place.

Dataset: CASME-II (255 spontaneous micro-expression clips)

- **CASME-II** — standard high-frame-rate (200 fps) spontaneous ME corpus.
- 255 clips total; I use a **3-class grouped** scheme and exclude the ambiguous *Others* bucket.
 - Negative = 99 · Positive = 32 · Surprise = 25 → pool of **156** clips.
- Severe **class imbalance** ($\approx 4 : 1.3 : 1$) — this single fact drives almost every result later.

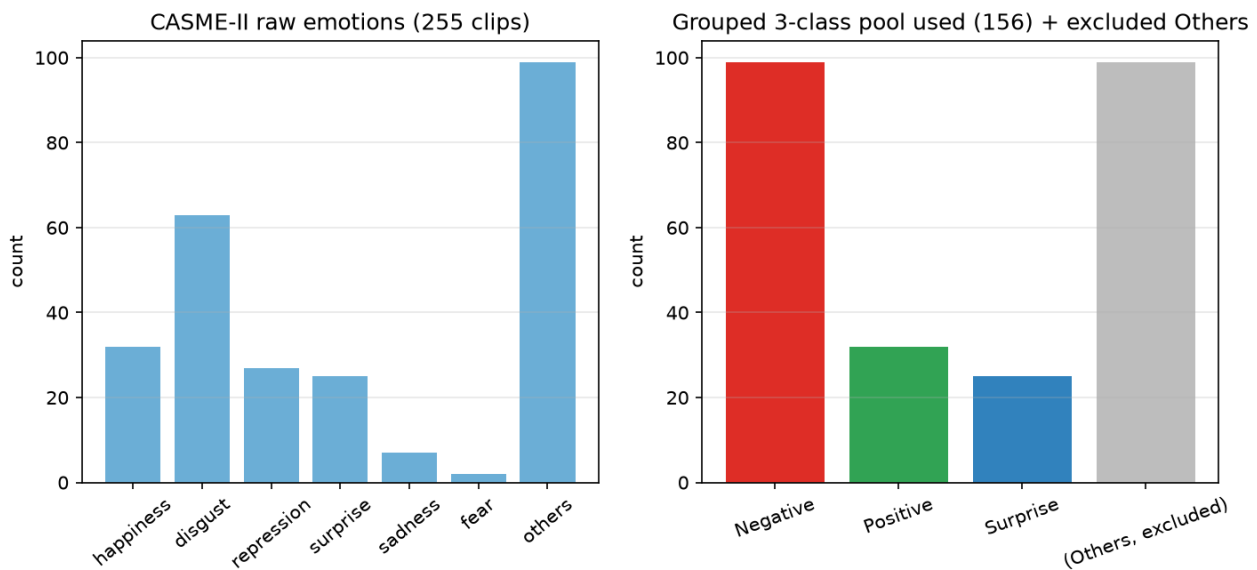


Fig. Dataset distribution: raw 7-way labels (left) and the grouped 3-class pool used (right).

► SPEAKER NOTES — what to say

This is the dataset: CASME-II, the standard benchmark for spontaneous micro-expressions, recorded at 200 frames per second so that the brief motion is actually captured. It has 255 labelled clips.

I grouped the raw emotions into three classes by valence — Negative, Positive and Surprise — and I deliberately excluded the 'Others' category, which is 99 ambiguous clips with no consistent emotion. That leaves a working pool of 156 clips: 99 Negative, 32 Positive, and only 25 Surprise.

I want you to notice the imbalance on this chart — roughly four-to-one between the largest and smallest classes. Please keep this in mind, because it is the single most important factor in interpreting my results. On a set this small and this skewed, plain accuracy becomes misleading, and a model can score well just by always guessing the majority class. That is why, throughout, my primary metric is macro-F1, which averages performance across all three classes equally.

System overview: a two-stage pipeline

- **Stage 1 — Data pipeline (offline motion modelling):**
 - Clip onset→offset, interpolate to a fixed **32 frames**.
 - Apply **EVM** (optional) → compute **optical flow (u,v)** + **optical strain** → tensor of shape (3, 32, 224, 224) .
- **Stage 2 — Neural network (motion + temporal modelling):**
 - **3D-CNN** spatial-temporal feature extractor → **SimAM** attention → **SLSTT transformer** over time → classifier.
- Every component is a **toggle** — that is what makes the ablation possible.
- Input channels carry *motion*; the 32-frame axis carries *time*.

► SPEAKER NOTES — what to say

Here is the whole system at a glance. It has two stages. Stage 1 is the data pipeline, and this is where the offline motion modelling happens: I cut each clip from the onset of the expression to its offset, resample it to a fixed length of 32 frames so everything is the same size, optionally apply Eulerian magnification, and then compute optical flow and optical strain. The output of Stage 1 is a tensor with three channels, 32 time steps, and 224-by-224 spatial resolution.

Stage 2 is the neural network, and this is where motion features get combined with temporal modelling. The 32-frame motion tensor goes into a 3D-CNN that extracts spatio-temporal features, then SimAM attention re-weights the most informative regions, then a sequence transformer models the long-range temporal dynamics, and finally a classifier outputs one of the three emotions.

The crucial design decision is that every one of these components is a switch I can turn on or off. That is what lets me run a controlled ablation later. And notice the clean separation of concerns: the three input channels encode *motion*, and the 32-step axis encodes *time* — motion and temporal modelling are literally built into the shape of the data.

Technology 1 — Eulerian Video Magnification (EVM)

- **Role:** *amplify* the micro-motion so the downstream flow/strain has a higher signal-to-noise ratio.
- Band-pass filters the temporal signal at each pixel and adds it back, scaled by a factor α :

$$I_{\text{mag}}(x, t) = I(x, t) + \alpha \cdot B\{I(x, t)\}$$

- $B\{\cdot\}$ = temporal band-pass filter; α = magnification factor.
- Implemented **offline** as a data-level switch: magnified tensors in `tensors/`, raw in `tensors_raw/`.
- *This is the first and most literal form of motion modelling — it makes invisible motion visible.*

► SPEAKER NOTES — what to say

The first technology, and the most literal piece of motion modelling, is Eulerian Video Magnification. The problem it solves is that the motion in a micro-expression is so small it is almost lost in sensor noise. EVM amplifies it.

The way it works, shown in the equation, is that for every pixel it looks at how the intensity changes over time, passes that temporal signal through a band-pass filter to isolate the frequency band where the micro-motion lives, multiplies it by a magnification factor alpha, and adds it back to the original. The result is a video where the subtle movement is visibly exaggerated, while everything static stays the same.

I implemented this offline, as a preprocessing step, not inside the network. So it becomes a simple data switch: when EVM is on, the pipeline reads from the magnified tensor folder; when it's off, it reads the raw folder. I'll flag now — and come back to it in the results — that this offline design is exactly where a data-routing bug crept in, which meant EVM never actually got exercised in my runs. The concept is sound; the verification step failed.

Technology 2 — Optical Flow + Optical Strain (the motion representation)

- **Role:** convert pixel changes into an explicit, identity-invariant **motion** description.
- **Optical flow** (u, v) — direction and magnitude of movement between frames.
- **Optical strain** — local *deformation* (skin stretch/compression around Action Units) that flow alone misses:

$$\epsilon_m = \sqrt{\epsilon_{xx}^2 + \epsilon_{yy}^2 + \frac{1}{2}(\epsilon_{xy}^2 + \epsilon_{yx}^2)}$$

- Stacked into the **3 input channels**: flow- u , flow- v , strain → robust to identity & lighting.
- *Why not raw RGB? Because RGB is dominated by identity; motion channels isolate the signal.*

► SPEAKER NOTES — what to say

The second technology turns raw movement into a clean, numerical motion representation. I use two complementary quantities. The first is optical flow, the standard u and v vectors that tell you the direction and speed of movement between consecutive frames. The second, and this is a deliberate choice, is optical strain.

Optical strain measures local *deformation* — how much the skin is stretching or compressing — which is computed from the spatial gradients of the flow field, as in the equation. This matters because facial Action Units, the muscle movements that define expressions, produce characteristic skin deformation that pure flow direction can miss. Strain captures the 'tightening' and 'bulging' of the face.

I stack flow- u , flow- v , and strain into the three input channels. The big advantage of feeding motion instead of raw RGB pixels is invariance: two different people performing the same micro-expression produce similar flow and strain, even though their faces look completely different and the lighting differs. On a dataset of only 156 clips, that sample-efficiency is essential — I cannot afford to make the network re-learn what a face looks like.

Technology 3 — 3D-CNN (spatio-temporal feature extractor)

- **Role:** learn local features that span **space and time together** — the bridge between motion and temporal modelling.
- Three parallel streams (one per input channel); 3D convolutions over (T, H, W) .
 - `cnn_out_channels` = 32×3 streams = **96** = the model width d_{model} .
- Captures *short-range* temporal structure (a few frames) plus spatial shape of the movement.
- When **off**, frames are average-pooled to 4×4 patches → a deliberately weak baseline.
- *This is where 'motion' (spatial pattern of flow) and 'time' (short windows) first fuse.*

► SPEAKER NOTES — what to say

The third technology is the 3D convolutional neural network, and it is the bridge between motion modelling and temporal modelling. A normal 2D CNN convolves over height and width. A 3D CNN also convolves over the *time* axis, so a single filter responds to a pattern that spans several frames — it is detecting *motion shapes*, not static edges.

I use three parallel streams, one for each input channel — flow-u, flow-v, and strain — and concatenate their outputs into a 96-dimensional feature vector per frame, which becomes the working width of the rest of the model. This captures short-range temporal structure, on the order of a few frames, together with the spatial layout of where on the face the movement is happening.

When I switch the CNN off for the ablation, I replace it with a deliberately crude baseline that just average-pools each frame into a tiny 4-by-4 grid. That gives me a control to measure exactly how much the 3D-CNN's spatio-temporal feature learning is worth. As you'll see in the results, it is worth a great deal — going from this weak baseline to the 3D-CNN is one of the biggest single jumps in performance.

Technology 4 — SimAM (parameter-free spatial attention)

- **Role:** focus the network on the *moving* region (the active Action Unit), suppress static background.
- Scores each neuron by how much it deviates from its feature map's mean/variance:

$$e_t^* = \frac{4(\sigma^2 + \lambda)}{(t - \mu)^2 + 2\sigma^2 + 2\lambda}, \quad \tilde{X} = X \odot \text{sigmoid}(1/e_t^*)$$

- **Adds ZERO learnable parameters** — critical on a 156-clip dataset (no extra overfitting risk).
- Only meaningful with the CNN on (it rescales CNN feature maps).
- *Best-value component: it sharpens motion features for free.*

► SPEAKER NOTES — what to say

The fourth technology is SimAM, a spatial attention module. Its job is to tell the network *where* to look — to emphasise the part of the face that is actually moving, the active muscle, and to suppress the static background and irrelevant regions.

What makes SimAM special, and why I chose it specifically, is that it is *parameter-free*. Instead of learning attention weights — which most attention modules do — it derives the importance of each neuron analytically, from how much that neuron stands out from the mean and variance of its own feature map, using the energy function on the slide. Neurons that are distinctive get amplified; bland ones get suppressed.

The phrase 'zero learnable parameters' is the whole point. On a dataset of only 156 clips, every extra parameter I add is an extra opportunity to overfit. SimAM gives me a genuine attention mechanism at no parameter cost whatsoever. It only makes sense when the CNN is on, because it rescales the CNN's feature maps. In my results it turns out to be the best-value component in the entire pipeline — it adds about twelve points of macro-F1 essentially for free.

Technology 5 — SLSTT Sequence Transformer (temporal modelling)

- **Role:** model **long-range** temporal dependencies across all 32 frames — the full onset→apex→offset trajectory.
- Pre-norm Transformer encoder: $d_{model} = 96$, 8 heads, 4 layers, with sinusoidal positional encoding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- Self-attention lets any frame attend to any other → captures timing/ordering the CNN cannot.
- When **off**, the time axis is collapsed by simple mean pooling.
- *This is the purest temporal-modelling component — and the one the experiments scrutinise hardest.*

► SPEAKER NOTES — what to say

The fifth and final technology is the sequence transformer — in my code the SLSTT, a Sequence-Level Spatio-Temporal Transformer. This is the purest temporal-modelling component. Where the 3D-CNN sees only a few frames at a time, the transformer looks at all 32 frames simultaneously and, through self-attention, lets any frame relate to any other frame.

In principle this is exactly what you want for micro-expressions, because an expression has a temporal trajectory — it onsets, peaks at an apex, and then offsets — and the transformer can model that ordering and timing directly. I use a four-layer, eight-head encoder with the standard sinusoidal positional encoding shown here, which injects the notion of frame order so the model knows which frame came first.

When I switch the transformer off for the ablation, I simply average the features over time, which throws away all ordering information. Now, here is the honest twist that the rest of the talk builds toward: although this is the most sophisticated and theoretically appropriate component, my experiments show it actually *hurt* performance on this small dataset. So this is the component my experiments scrutinise most carefully, and understanding *why* it failed is a key contribution of the thesis.

My final model: combining everything (Config 8 — 'Proposed Unified')

- My final experiment chains **all five technologies** end to end:
 - **Motion:** EVM → Optical Flow + Strain → 3D-CNN
 - **Attention:** SimAM on the CNN feature maps
 - **Temporal:** SLSTT Transformer over the 32 frames → classifier
- Trained with **class-weighted Focal Loss** to fight the imbalance.
- This 'everything-on' model is **Config 8**, the proposed unified architecture the thesis set out to validate.
- The ablation then asks: *does combining everything actually beat the simpler combinations?*

► SPEAKER NOTES — what to say

This slide shows my final, complete model — the one the thesis set out to build — which I call Config 8 or the Proposed Unified model. It chains together all five technologies in one forward pass.

Reading it as motion-then-time: on the motion side, a clip is magnified by EVM, converted into optical flow and strain, and passed through the 3D-CNN to extract spatio-temporal features. SimAM attention then sharpens those features by focusing on the moving region. On the temporal side, the transformer models the long-range dynamics across all 32 frames, and a final classifier produces the emotion. It is trained with class-weighted focal loss, which I'll explain next, specifically to cope with the imbalance.

So this is 'everything switched on'. But the scientific question of the whole thesis is precisely this: does combining everything actually beat the simpler combinations? It is tempting to assume the most complex model wins. The ablation study I describe next is designed to test that assumption rather than take it on faith — and the answer turns out to be surprising.

Training setup & the imbalance defence

- **Class-weighted Focal Loss** down-weights easy majority examples, focuses on hard minority ones:

$$\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t), \quad \gamma = 2.0$$

- Plus `label_smoothing = 0.05` and inverse-frequency class weights.
- Optimiser: Adam, `lr = 1e-4`, `weight_decay = 1e-4`, **60 epochs**, gradient clipping at 1.0, mixed precision.
- `batch_size = 2` — a hardware limit (8 GB VRAM) that also causes training noise (shown later).
- Checkpoint selected on best **validation macro-F1**, not accuracy.

► SPEAKER NOTES — what to say

Before the experiments, a word on training, because the imbalance forced specific choices. My loss function is class-weighted focal loss. Focal loss, with the gamma of 2 shown here, automatically reduces the weight of examples the model already classifies confidently — which are mostly the majority Negative class — and concentrates the learning signal on the hard, rare minority examples. On top of that I add inverse-frequency class weights and a little label smoothing. All three of these are imbalance defences.

For optimisation I use Adam with a small learning rate, weight decay for regularisation, 60 epochs, gradient clipping for stability, and mixed precision to save memory. One number to flag is the batch size of just 2. That was forced by GPU memory — these 32-frame, 224-by-224, three-channel tensors are large — and I'll show later that this small batch is partly responsible for noisy, unstable training curves.

Finally, and importantly, I select the best model checkpoint based on validation macro-F1, not accuracy or loss. Because accuracy is misleading under imbalance, choosing the checkpoint by macro-F1 makes sure I keep the model that is genuinely balanced across all three classes.

Experimental design — the ablation matrix (12 configs)

- Four independent on/off switches: **EVM, SimAM, 3D-CNN, Transformer** → 12 valid combinations.
- Each row is one experiment; ✓ = technology used, ✗ = not used. **Config 8 = my full model.**

Config	EVM	SimAM	3D-CNN	Transf.	Role
C1 pure base	✗	✗	✗	✗	lower bound
C2 temporal	✗	✗	✗	✓	transformer only
C3 spatial	✗	✗	✓	✗	CNN only
C5 attention	✗	✓	✓	✗	CNN+SimAM
C6 full no-EVM	✗	✓	✓	✓	Stage-2 full
C8 proposed	✓	✓	✓	✓	FINAL MODEL

► SPEAKER NOTES — what to say

This slide is the experimental design — the heart of the methodology. I have four independent switches: EVM, SimAM, the 3D-CNN, and the transformer. Turning them on and off in all valid combinations gives twelve configurations. The table shows the key ones; a tick means that technology is used in that experiment, a cross means it isn't.

Read it as a ladder of complexity. Config 1 is the pure baseline with everything off — my lower bound. Config 3 adds just the 3D-CNN. Config 5 adds SimAM on top of the CNN. Config 6 adds the transformer too. And Config 8, highlighted in yellow, is my final proposed model with every technology switched on, including EVM.

The beauty of this design is that comparing any two adjacent rows isolates the effect of exactly one technology. So when I compare Config 5 to Config 8, I am measuring precisely what the transformer adds. This is what lets me make evidence-based claims about each component instead of just asserting that my big model is good. I ran this whole matrix in both the 3-class and a harder 6-class labelling, and under several validation protocols, which I'll cover next.

Experiment runs — four validation regimes (git branches)

- The same ablation matrix was evaluated under increasingly strict protocols:

Branch / run	Protocol	Test N	Configs	Purpose
main	Holdout (1-epoch smoke)	39	1	wiring check
holdout-all	Holdout, 60 epochs	39	12	primary screen
loso-handle	Pilot LOSO (5 folds)	24	8	fast cross-val
full-loso	Mixed / interrupted	39 & 24	12	attempted full LOSO

- Holdout** = one subject-disjoint split (fast). **LOSO** = leave-one-subject-out (gold standard).

► SPEAKER NOTES — what to say

I didn't run the ablation just once — I ran it under four different validation regimes, and each one lives on its own git branch, which gives me a clean audit trail. They go from cheap and approximate to strict and expensive.

The 'main' branch is just a one-epoch smoke test to prove the plumbing works end to end. 'holdout-all' is my primary result: all twelve configs, trained for 60 epochs, evaluated on a subject-disjoint holdout split of 39 clips. 'loso-handle' is a pilot leave-one-subject-out with five folds, to cross-validate faster. And 'full-loso' was meant to be the full gold-standard leave-one-subject-out over all subjects.

I'll be transparent here, because honesty about methodology is part of good science: the full-LOSO branch never completed cleanly — most of its configs actually report holdout-sized results, and a few are leftover pilot runs. So my soundest, most complete evidence comes from the holdout-all branch, and I treat the LOSO numbers as supporting, not definitive. The difference between holdout and LOSO matters: holdout is one split and is optimistic, whereas leave-one-subject-out tests true generalisation to unseen people, which is the gold standard the field expects.

Experiment procedure — step by step

- **Step 1.** Parse CASME-II metadata → unified label CSV (255 clips).
- **Step 2.** Clip onset→offset, interpolate to 32 frames.
- **Step 3.** (Motion) compute optical flow + strain, with/without EVM → (3, 32, 224, 224) tensors.
- **Step 4.** For each of the 12 configs, assemble the network from its on/off switches.
- **Step 5.** Train 60 epochs with focal loss; pick best checkpoint by val macro-F1.
- **Step 6.** Evaluate on the held-out subjects → accuracy, macro-F1, confusion matrix, per-class F1.
- **Step 7.** Aggregate all configs into `summary.csv`; plot figures; compare.

► SPEAKER NOTES — what to say

Here is the actual procedure, step by step, so you can see exactly how a result is produced. Step one: I parse the CASME-II coding spreadsheet into a single unified label file of 255 clips. Step two: for each clip I cut from the onset frame to the offset frame and interpolate to a fixed 32 frames.

Step three is the motion-modelling step: I compute optical flow and optical strain — optionally on EVM-magnified frames — and save each clip as a three-channel, 32-frame, 224-by-224 tensor. Step four: for a given configuration, I assemble the network by reading its four switches and including or excluding each module. Step five: I train for 60 epochs with focal loss and keep the checkpoint with the best validation macro-F1.

Step six: I evaluate that best model on the held-out subjects and record accuracy, macro-F1, the confusion matrix, and per-class F1. Step seven: I repeat for all twelve configs, collect everything into a summary file, and generate the comparison plots you're about to see. Every number in the results section comes out of this exact pipeline — nothing is hand-tuned or cherry-picked.

Results 1 — Holdout: accuracy is flat, macro-F1 is what matters

- Accuracy sits at 0.72–0.77 for every config; macro-F1 ranges **0.28 → 0.74**.
- **Best: Config 5 (SimAM + 3D-CNN), macro-F1 = 0.7427** — clears the 0.68 target.

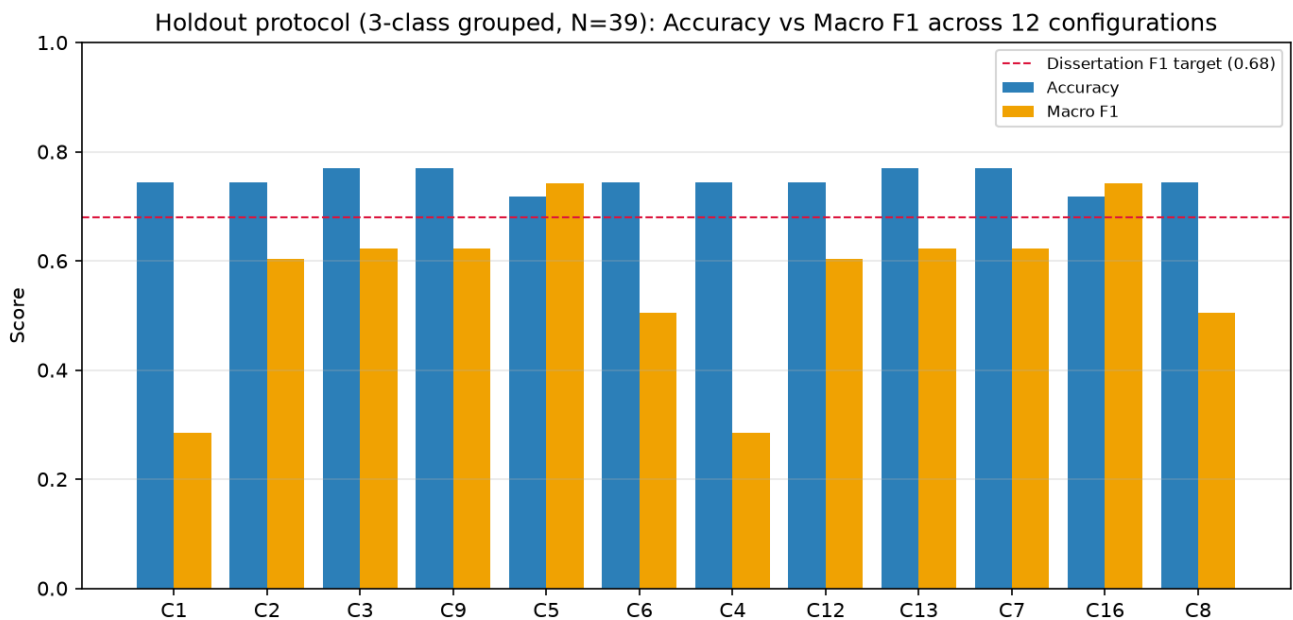


Fig. 1 — Accuracy (blue) vs macro-F1 (amber), 12 configs, Holdout. Only C5/C16 beat the 0.68 target (red line).

► SPEAKER NOTES — what to say

Now the results. This first chart shows all twelve configurations on the holdout protocol, with accuracy in blue and macro-F1 in amber, and the red dashed line is my 0.68 target.

The first thing to see is that accuracy, the blue bars, is almost flat — every model scores between 0.72 and 0.77 — because they can all get the majority class right. If I'd reported only accuracy, I'd wrongly conclude every model is equally good. The amber bars, macro-F1, tell the real story: they range from 0.28 all the way to 0.74. That spread is where the science is.

The best model is Config 5 — that's SimAM plus the 3D-CNN, with no transformer and no EVM — at a macro-F1 of 0.7427, comfortably above my target. Take a moment on which model that is: it is *not* my full proposed model. It's a motion-modelling model with attention but *without* the temporal transformer. The two best bars, C5 and C16, are exactly the ones that have the CNN and SimAM but no transformer. That's the first strong hint that the temporal transformer isn't helping.

Results 2 — why the transformer hurts: it kills the minority class

- Comparing the best model (C5) vs my full model (C8) isolates the **transformer's** effect.
- C8 routes **all 9 Positive clips into Negative** → Positive recall = 0, macro-F1 drops 0.74 → 0.51.

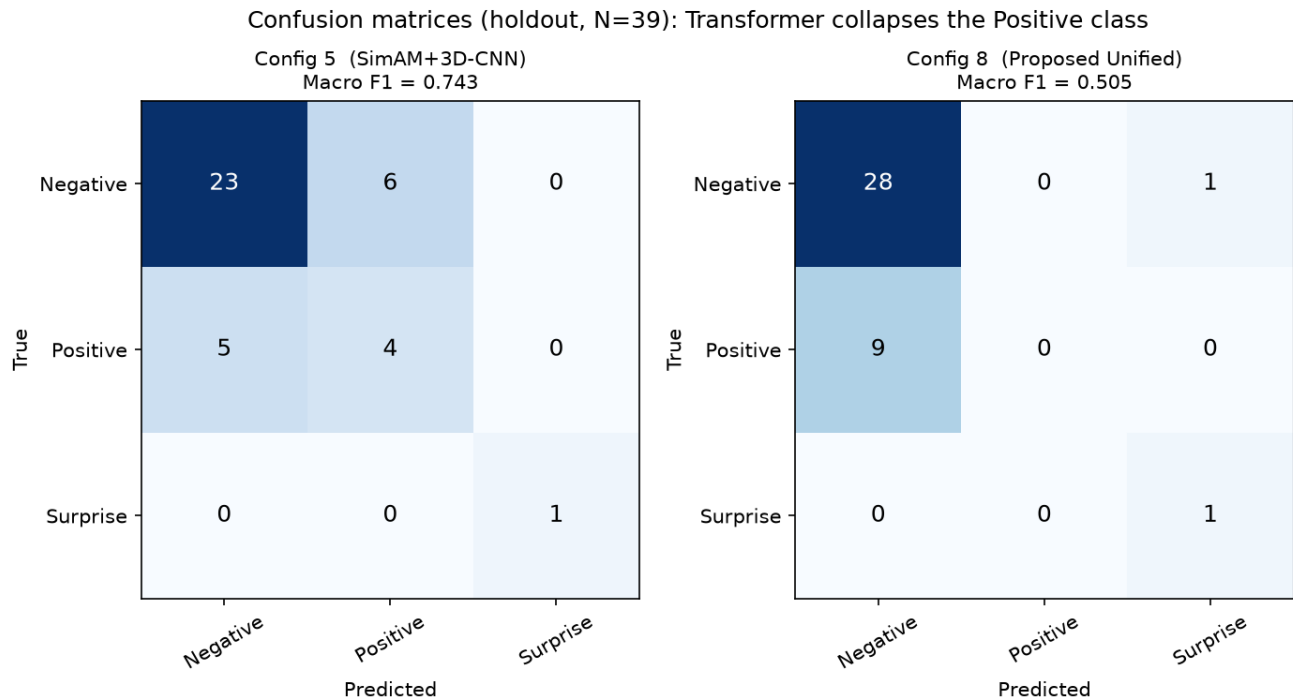


Fig. 2 — Holdout confusion matrices. Left C5 recovers Positives; right C8 (with transformer) empties the Positive column.

► SPEAKER NOTES — what to say

This is the most important results slide, because it explains *why* the transformer hurts. I compare my best model, Config 5, on the left, against my full proposed model, Config 8, on the right. The only meaningful difference between them is the transformer, so this comparison isolates its effect.

Look at the confusion matrices. On the left, Config 5 genuinely separates the classes — it correctly recovers four of the nine Positive clips and the single Surprise clip. On the right, Config 8, with the transformer added, the entire Positive column is empty: all nine Positive clips are misclassified as Negative. Its Positive recall is literally zero. That's why its macro-F1 collapses from 0.74 down to 0.51, even though its raw accuracy actually went *up*.

The mechanism is overfitting through excess capacity. The transformer adds tens of thousands of parameters, but I'm training on only about 117 clips with a batch size of 2. With so few minority examples, the lowest-loss shortcut the transformer finds is to just predict the majority class almost everywhere and accept the penalty on the rare classes. SimAM avoids this trap precisely because it adds no parameters. So the lesson is concrete: temporal modelling with a high-capacity transformer needs far more data than CASME-II provides.

Results 3 — per-class F1 and the EVM anomaly

- Per-class F1: models diverge almost entirely on the **minority Positive** class.
- **EVM anomaly:** every EVM-on config is *identical* to its EVM-off twin → EVM was never actually exercised (a data-routing bug).

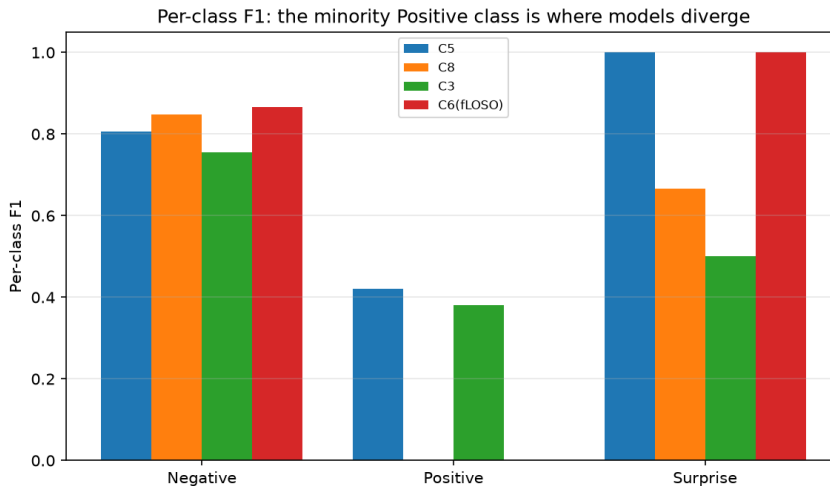


Fig. 3 — Per-class F1: transformer models score 0.00 on Positive.

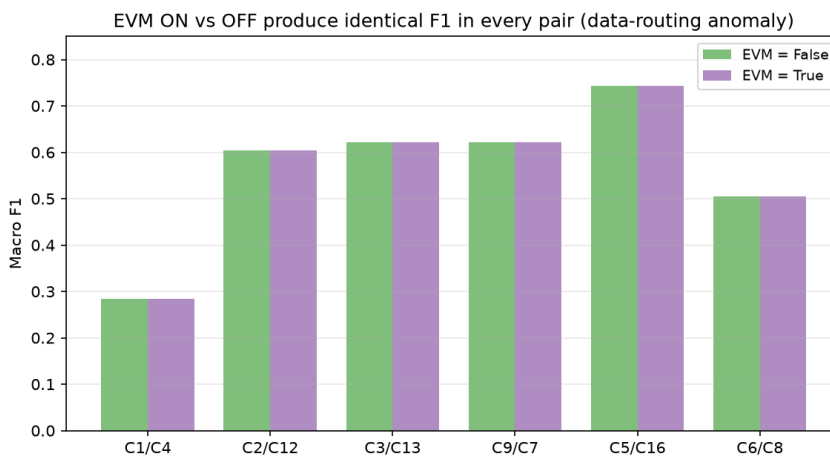


Fig. 4 — EVM-on equals EVM-off in all six pairs.

► SPEAKER NOTES — what to say

Two more results here. The left chart breaks performance down by class, and it confirms the previous slide: the models barely differ on Negative and Surprise — they diverge almost entirely on the minority Positive class. The full transformer models, the rightmost bars, score a flat zero on Positive. So the whole performance story is really a story about whether a model can hold onto the rare Positive class or not.

The right chart documents an anomaly I found, and I report it openly because it affects how you should read my EVM conclusions. Every configuration with EVM switched on produced results *identical to four decimal places* to the same configuration with EVM off — all six pairs are exactly equal. That cannot happen by chance. It means both settings were reading the same input data: a data-routing bug, or a zero-magnification setting, caused EVM to never actually be applied.

The honest consequence is that my EVM hypothesis is *untested*, not disproven. The motion-magnification idea is sound in the literature; my pipeline just failed to exercise it, and verifying and re-running EVM is my number-one piece of future work. I'd rather tell you this than quietly present duplicated numbers as if they validated EVM.

Results 4 — protocol sensitivity & training stability

- Pilot-LOSO inflates accuracy to 0.98 but macro-F1 stays < 0.45 — classic imbalance artifact on tiny folds.
- Config 8 training is highly **unstable** (val-F1 swings 0.05-0.74) — small batch + over-capacity.

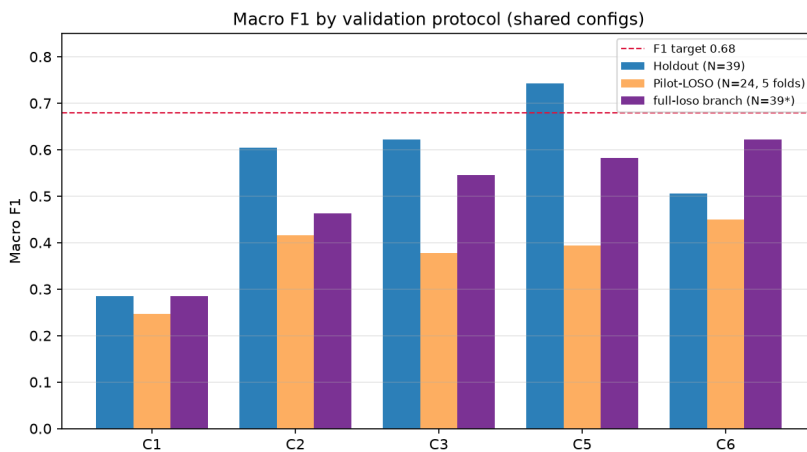


Fig. 5 — Macro-F1 by protocol.

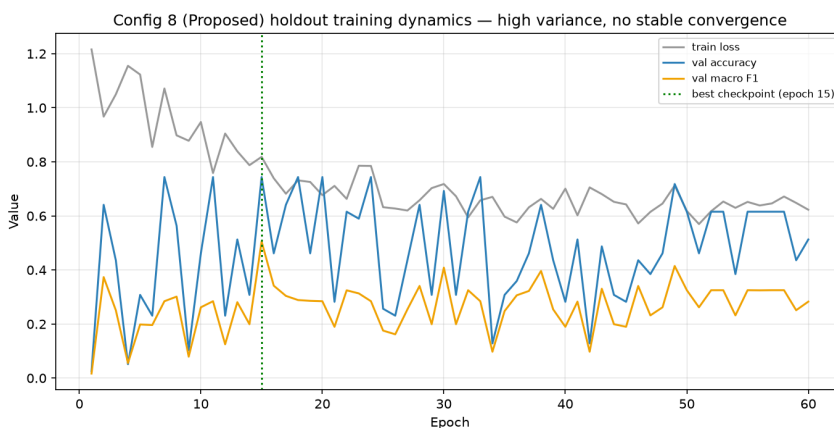


Fig. 6 — Config 8 unstable training curves.

► SPEAKER NOTES — what to say

These two charts cover robustness. On the left I compare macro-F1 across the validation protocols. The key warning is about the pilot-LOSO: it produced eye-catching accuracies up to 98%, but its macro-F1 stayed below 0.45 for every config. That gap is a pure artifact of imbalance on tiny folds — when a fold contains almost only Negative clips, predicting Negative everywhere scores 98% accuracy but fails completely on the absent classes. It's a cautionary tale about trusting accuracy.

On the right is the training curve for my full Config 8 model. Notice how violently the validation accuracy and F1 oscillate from epoch to epoch — F1 bounces between 0.05 and 0.74 and never settles. That instability is a direct symptom of two things I mentioned earlier: the batch size of only 2, which makes each gradient estimate noisy, and the transformer's excess capacity on too little data.

Together these tell me that the full model isn't just slightly worse — it's genuinely unstable, bouncing between majority-collapse states rather than converging on a balanced solution. So future work needs gradient accumulation for a larger effective batch, and multiple seeds reported with error bars, before any temporal-transformer claim can be considered solid.

Comparison table — technologies vs results across all configs

- ✓ = technology used, ✗ = not used. Sorted by Holdout macro-F1. **My final model (C8) highlighted.**

Config	EVM	SimAM	CNN	Transf.	Acc	Macro-F1
C5 attention base	✗	✓	✓	✗	0.718	0.743
C16 (= C5 +EVM)	✓	✓	✓	✗	0.718	0.743
C3 spatial only	✗	✗	✓	✗	0.769	0.622
C9 / C7 CNN+Transf	✗	✗	✓	✓	0.769	0.622
C2 / C12 transf only	✗	✗	✗	✓	0.744	0.604
C8 PROPOSED (all on)	✓	✓	✓	✓	0.744	0.505
C6 full no-EVM	✗	✓	✓	✓	0.744	0.505
C1 / C4 pure base	✗	✗	✗	✗	0.744	0.284

► SPEAKER NOTES — what to say

This is the comparison table you asked for, with a tick for every technology used and a cross for every one left out, sorted from best to worst macro-F1 on the holdout protocol, and my final proposed model highlighted in yellow.

Reading top to bottom tells the whole thesis in one table. The top two rows — the winners at 0.743 — both have SimAM and the CNN but a cross under the transformer. Then the CNN-only and CNN-plus-transformer configs sit around 0.62. My full proposed model, C8 with everything ticked, is well down the list at 0.505, tied with C6 — the only difference between those two is the inert EVM column, which is why they're identical. And the pure baseline with everything crossed off is dead last at 0.284.

Two patterns jump out. First, the EVM column makes no difference anywhere — every EVM pair has the same score, the anomaly again. Second, and most importantly, adding the transformer's tick *lowers* the score: the all-ticked model loses to the simpler SimAM-plus-CNN model by over twenty points of macro-F1. So 'more technology' did not mean 'better' — and being able to demonstrate that cleanly, with this table, is exactly the value of having run a full ablation.

What I built and combined — and what actually won

- **My final combined experiment (Config 8)** integrates all five technologies: EVM + Flow/Strain + 3D-CNN + SimAM + Transformer.
- **What actually won:** *motion modelling + attention* — **Config 5 (Flow/Strain → 3D-CNN → SimAM)**, macro-F1 **0.743**.
- **Temporal modelling verdict:** the 3D-CNN's short-range temporal convolution helps; the transformer's long-range modelling **overfits** on 156 clips and must be earned with more data.

Component (final model)	Type	Used	Verdict
EVM	Motion	✓	untested (bug)
Optical flow + strain	Motion	✓	essential
3D-CNN	Motion+Temporal	✓	biggest gain
SimAM	Attention	✓	best value (free)
SLSTT Transformer	Temporal	✓	hurt — needs more data

► SPEAKER NOTES — what to say

Let me synthesise. My final experiment combined all five technologies into the proposed unified model — EVM, optical flow and strain, the 3D-CNN, SimAM, and the transformer — and that is the complete system the thesis set out to build and validate.

But the experiment's verdict, which is the real contribution, is more interesting than 'the big model works'. What actually won was the combination of motion modelling and attention: Config 5, which is flow and strain into a 3D-CNN with SimAM, at a macro-F1 of 0.743, above target. The summary table grades each component of my final model: optical flow and strain are essential; the 3D-CNN gives the biggest single gain and provides short-range temporal modelling; SimAM is the best-value component because it helps for free; EVM is untested because of the bug; and the transformer, despite being the most advanced temporal model, actually hurt.

So on my thesis title — temporal and motion modelling — the nuanced answer is this: motion modelling clearly works, and short-range temporal modelling via the 3D-CNN works, but high-capacity long-range temporal modelling with a transformer is premature on a dataset this small. It has to be *earned* with more data, not assumed.

Conclusion & future work

- **Conclusion:** combining motion modelling (flow/strain + 3D-CNN) with parameter-free SimAM attention gives the best, most generalisable result (macro-F1 0.743), beating the full transformer model.
- Accuracy is misleading under imbalance — **macro-F1 must be the headline metric.**
- **Future work:**
 - Fix & re-run **EVM** (verify magnified $\neq$ raw tensors).
 - Run a true **full 26-subject LOSO** with multiple seeds + error bars.
 - Shrink / pre-train the **transformer**, or add data (SAMM, SMIC) before trusting long-range temporal modelling.
 - Gradient accumulation for a larger effective batch → stable training.
- **Contribution:** an honest, ablation-driven account of *which* temporal and motion components actually improve MER.

► SPEAKER NOTES — what to say

To conclude. My main finding is that, for micro-expression recognition on CASME-II, the winning recipe is motion modelling — optical flow and strain through a 3D-CNN — combined with parameter-free SimAM attention. That combination reaches a macro-F1 of 0.743 and beats my own full transformer-based model. A secondary but practically important finding is methodological: under class imbalance, accuracy is deceptive and macro-F1 has to be the headline metric.

For future work, in priority order: first, fix and re-run EVM, with an explicit check that the magnified and raw tensors actually differ, so the motion-magnification hypothesis can finally be tested. Second, run a proper full leave-one-subject-out over all subjects, with multiple random seeds and error bars, so the comparisons are statistically solid. Third, before trusting the temporal transformer, either shrink it, pre-train it, or add more data from datasets like SAMM and SMIC. And fourth, use gradient accumulation to get a larger effective batch and stabilise training.

The contribution of this thesis is not a single magic number — it is an honest, ablation-driven account of *which* temporal and motion-modelling components genuinely improve micro-expression recognition, and which ones look good on paper but don't pay off yet. Thank you — I'm happy to take questions.